Browser lifetime + stop rule: test results (release candidate `9a1b931`)

- Branch: `fix/browser-lifetime-and-stop-rule`, base `b676c2d`
- **Release candidate:** **`9a1b931`** (code + tests). It is built from the fix commit `f2288ce`, the tests/harness commit `caaa191`, and the release-item commit `9a1b931`, all pushed to origin.
- The §1 repro and the close-order mutation check were measured at `f2288ce` / `caaa191`; `9a1b931` does not touch `processor.ts`. All test suites in §2–§5 were re-run on `9a1b931`.
- Run on 2026-10-12, on this VM (2 vCPU). No Railway run, no paid run, no real LLM calls.
- Evidence files:
- `evidence/browser-lifetime-candidate/`: repro JSON and logs.
- `evidence/regression-caaa191/`: previous candidate's suite logs and tsc.
- `evidence/release-items/`: stop-rule unit + real-PG logs, integration identity runs, `tsc.txt`, and `regression/` with every suite log on `9a1b931`.

1. Before/after repro (server/scripts/browser-lifetime-repro.ts)

Setup. `WORKER_ISOLATION_ENABLED=true`, worker concurrency 4, `MAX_CONCURRENT_BROWSER=1`, 12 jobs, 1 document per child, 20 s synthetic scoring hold after the fetch. Each document is a synthetic PDF with its own text marker. Each run checks that every document was requested, extracted with its marker, and never skipped as BrowserUnavailable.

How the runs were made. Each side ran in its own git worktree: `/tmp/wt-base` at `b676c2d` and `/tmp/wt-cand` at `f2288ce`. Both used the same harness files, copied from `caaa191`, so only `processor.ts` / `worker.ts` differ. Runner: `evidence/browser-lifetime-candidate/wt-run.sh`. The TasksMax runs used `systemd-run --scope -p TasksMax=400`, i.e. a cgroup pids limit of 400.

Metric	Base, unlimited	Candidate, unlimited	Base, TasksMax=400	Candidate, TasksMax=400
Peak live browsers (distinct profiles)	**4**	**1**	**4**	**1**
Live-browser bound (≤ permits) held	no	yes	no	yes
Peak tasks (threads)	550	205	367	180
Peak processes	42	19	29	16
Peak summed RSS (MB, upper bound)	4,941	2,048	3,398	1,954
Jobs ok / failed	12 / 0	12 / 0	**8 / 4**	12 / 0
Docs extracted with marker	12/12	12/12	**8/12**	12/12
Chromium launches	12	12	15	12
“Failed to launch the browser process” lines	0	0	6	0
Wall time	126.3 s	127.4 s	**501.9 s**	127.6 s
Admission wait, mean / p95	6.1 s / 24.7 s	6.4 s / 24.5 s	93.2 s / 136.4 s	6.5 s / 24.9 s
Permit hold, mean / p95	8.3 s / 8.6 s	8.4 s / 8.6 s	39.6 s / 126.5 s	8.4 s / 8.6 s
Per-document fetch, mean	14.4 s	14.9 s	132.8 s	14.9 s
Final broker permits active	0	0	0	0
Exit	0	0	0	0

Reading the table

- **Unlimited.** The base keeps one Chromium alive per child until that child exits. That gives 4 live browsers with 1 permit. The candidate never has more than 1. Threads fall by about 63% (550 → 205) and summed RSS by about 59%. Wall time, admission wait and permit hold are unchanged. In **this** workload (1 document per child, local PDFs, 1 permit), closing the browser before releasing the permit added no measurable latency (permit hold 8.3 s vs 8.4 s). That is not a general zero-overhead claim: a relaunch per permit cycle has a cost that depends on fetch mix and concurrency, and it was not measured at production settings.
- **TasksMax=400.** This cgroup pids limit makes the base fail the same way production does, as browser-launch failures:
- Puppeteer printed 6 “Failed to launch the browser process … Code: null” lines and 3 “Network.enable timed out” lines, and one fetch hit “Timed out after waiting 30000ms”.
- 4/12 jobs ended with `no document text`. Their PDFs were never requested (`neverRequested` = job-1002/1005/1008/1011). Admission wait rose to a mean of 93 s, and the run took 3.9× as long.
- The candidate under the same limit stays at 1 browser, 180 tasks and 12/12 jobs, the same as its unlimited run.
- **Caveat.** The production SIGABRT signature was **not** reproduced locally. What reproduced is the launch-failure / starvation mode under a task cap. The fix removes the demonstrated browser accumulation and the extraction failures that came with it in this constrained repro. Production must still show whether it eliminates the child-start failures. The harness labels a failed fetch “fetch ok … chars=0 marker=false” in the child log line. The summary `extraction` block is the authoritative per-document check.

The earlier repro evidence in `evidence/browser-lifetime/` was made in the main checkout before the worktree method, with the old counter. It is superseded by this table.

2. New tests

Suite	Result	What it proves
`server/lib/browser-close-order.test.ts`	**4/4 pass** (56 s)	On the success, fetch-failure (direct + pdfRecovery), cancellation (in-flight / post-acquire refusal / pre-acquire refusal) and child-death paths, the child's Chromium process group is gone **before** the browser permit returns to the parent broker. On the child-death path, the parent reaps before it releases.
`server/lib/batch-stop-rule.test.ts`	**20/20 pass** with `STOP_RULE_TEST_PG` (disposable local PG); 19 + 1 skip without it	Rule logic, gating, the batch-1276 shape, and the durable-first cancel sequence with never-resolving and failing purge/snapshot fakes (§4). Log: `evidence/release-items/batch-stop-rule.unit.log`
`server/lib/batch-stop-rule.db.test.ts`	**17/17 pass**	Real-PG stop rule and concurrent-failure race on a throwaway DB (§4). Log: `evidence/release-items/batch-stop-rule.db.log`

Mutation check (from the earlier session, same code). Releasing the permit before the reap check makes the close-order tests fail: more than one live Chromium at permit return. So the ordering is what the tests detect, not an accident.

3. Regression suites

Each suite runs with `DATABASE_URL=postgres://stub:stub@127.0.0.1:1/stub npx tsx server/lib/<suite>.test.ts`, re-run on `9a1b931`. Logs are in `evidence/release-items/regression/`; the previous run at `caaa191` is in `evidence/regression-caaa191/`.

Suite	Result	Exit
job-isolation	29/29	0
lifecycle-fence.unit	42/42	0
scoring-cancellation.unit	60/60	0
credit-breaker	22/22	0
llm-usage	20/20	0
adaptive-concurrency	13/13	0
runtime-diag.unit	21/21	0
reconcile-resurrect	15/15	0
reliability	PASS (20 acceptance checks)	0
provider-resilience	112/112	0
transport-abort.unit	55/55	0
browser-close-order (new)	4/4	0
snapshot-durability	12/12	0
lifecycle-fence.db (real local PG/Redis; includes S7B concurrent `cancelBatchRun`)	165/165	0
batch-stop-rule (new)	20/20 (with local PG)	0
batch-stop-rule.db (new, real local PG/Redis)	17/17	0

The counts for existing suites match the previous branch baseline (`TEST_RESULTS` for `fix/worker-isolation`).

Integration suite (opt-in, real Postgres/Redis/BullMQ/Chromium)

`DATABASE_URL=postgres://ciq:ciq@127.0.0.1:55432/ciq REDIS_URL=redis://127.0.0.1:56379 INT_REQUIRED=1 npx tsx server/lib/job-isolation.integration.test.ts` (local docker `postgres:16-alpine` + `redis:7-alpine`, concurrency 2):

Run	Result	Peak Chromium under job children
Base `b676c2d` (worktree, previous session)	PASS 11/11	18 procs, 2,398 MB
`caaa191` run 1 (previous session)	**FAIL 2/11** (extra process, unidentified then)	10 procs, 1,381 MB
`caaa191` run 2 (previous session)	PASS 11/11	10 procs, 1,377 MB
`9a1b931` attempt 1	**FAIL 6/12**: not a product failure, see below	0 (no job ran)
`9a1b931` attempt 2 (warm tsx cache)	PASS 12/12	see log
`9a1b931` attempt 3 (`TSX_DISABLE_CACHE=1` , **final**)	**PASS 12/12**	10 procs, 1,373 MB

Identity of the extra process (release item 2). The integration test now records exe (readlink `/proc/<pid>/exe`), cmdline, ppid, comm, start time and RSS for every descendant. It does this before the jobs start, in every sample, and in every failure message. The previous “baseline descendants excluded” mechanism was removed. In the final run (`evidence/release-items/integration-identity.{log,json}`), the extra process was:

Field	Value
exe	`node_modules/.pnpm/@esbuild+linux-x64@0.28.0/node_modules/@esbuild/linux-x64/bin/esbuild` (in the repo)
cmdline	`<exe> --service=0.28.0 --ping`
parent	the test harness process itself (pid 139822; the helper is pid 139834)
existed before the jobs?	**Yes.** It appears in the pre-job descendant snapshot taken right after `initializeDatabase` and before any job was enqueued, and in all 98 samples.
app work or toolchain?	**Toolchain.** It is tsx's esbuild transform service, which tsx spawns lazily the first time it must compile uncached TypeScript. With a warm tsx cache (attempt 2) it was never spawned (pre-job descendants `[]`). With the cache disabled (attempt 3) it was. Production runs compiled JS without tsx. The job children are separate: 2 × `/usr/bin/node --import tsx …/server/lib/job-child.ts` at peak.
exits with the harness?	**Yes.** The post-exit check found "pid 139834 gone; any `esbuild --service` processes on host: none".
~RSS	12–15 MB, steady

This is consistent with `caaa191` run 1: one non-Chromium direct child of the test process at ~14 MB, present from the first sample through `stopWorker`. In that run the helper was evidently spawned after the baseline snapshot, so the timing-sensitive baseline exclusion missed it. The identity was not recorded in run 1 itself; it is attributed from the matching footprint and from the identities recorded in this session's runs.

Fix in the test: explicit allowlist. The check was not relaxed.

- A descendant is excluded only if all of these hold: it is a direct child of the test pid, its exe basename is `esbuild`, and its cmdline matches `esbuild --service=<ver> --ping`. Each exclusion is recorded in `toolchainHelpers` and printed.
- Every other descendant is still counted. All zero-descendant assertions are unchanged ("no descendants after completion", "no descendants after stopWorker", "no orphaned puppeteer Chromium anywhere on host"). The concurrent-children check still requires exactly 2.

`9a1b931` attempt 1 (FAIL 6/12): self-inflicted, not a product failure.

- My first version of `batch-stop-rule.db.test.ts` left its Redis cross-replica cancel flag `cancelled:batch:9` behind. The throwaway DB is cloned from the same template, so the next integration run also got batch id 9.
- The worker therefore treated that batch as cancelled and never claimed its jobs. The six failing checks are job/batch/score/children/Chromium/lock-renewal (`pending,pending` , `max seen 0`). The process-hygiene checks passed.
- The flag was deleted, and the db test's teardown now deletes it and verifies that the throwaway DB is gone.
- Log: `evidence/release-items/integration-identity-attempt1-redis-polluted.log` . It also records the same esbuild helper as a pre-job descendant (pid 138434).

The fix also shows in the integration run: peak Chromium under 2 concurrent children fell from 18 to 10 processes (one browser at a time instead of one per child). Summed RSS fell from 2,398 to ~1,380 MB.

4. Stop rule S4 (`BATCH_STOP_RULE_FAILED_JOBS`)

The rule is off when the variable is unset, 0 or junk (test 1). With the variable unset, the worker's retry path is unchanged from the legacy one: the rule never counts and never cancels (test 9).

Scenario (batch-1276 shape)	Outcome
Rule off	13 jobs exhaust retries (39 failed attempts), as happened in batch 1276
`BATCH_STOP_RULE_FAILED_JOBS=2`	Batch cancelled at the **2nd** failed job, before any job exhausts retries
N=3	The 3rd failed job trips the rule. The batch is cancelled, a snapshot saved and the queue purged, with no further retries or claims

Other behaviour:

- Cancellations and provider-quota pauses do not count as S4 failures.
- If the failure count errors, the rule fails open to the legacy retry path.
- A cancel-sequence error still suppresses the retry.

Durable-first cancel (release item 1). The sequence runs in this order:

1. The flag (local + Redis notification), synchronously.
2. The PG cancel and the queue purge start together. The purge is bounded at 10 s.
3. Only if the PG cancel committed and transitioned the batch: a snapshot bounded at 30 s, marked `rejected` / "cancelled/partial … not comprehensive".

Unit tests 12–19 use never-resolving and rejecting fakes, with 300 ms bounds:

Fake purge / snapshot	PG cancel committed	Sequence settles
hangs / ok	< 100 ms	within the bound (< 2 s)
ok / hangs	< 100 ms	within the bound
hangs / hangs	< 100 ms	within the bound
rejects / rejects	< 100 ms	within the bound
hangs / rejects	< 100 ms	within the bound

- Test 17 observes the PG cancel resolved while the purge is still hung.
- Test 18: a PG-cancel error propagates after the bounded purge, and no snapshot is taken.
- Test 19: an already-cancelled batch (`cancelBatchRun` → false) takes no second snapshot.
- In all of these the flag is set synchronously, before any await.

Real-PG stop-rule test (release item 3), `server/lib/batch-stop-rule.db.test.ts`: 17/17 PASS.

- **Safety.** The test refuses to run unless the host is loopback and the database name does not contain “companyiq”. It ran against a throwaway database `stoprule_<ts>` created from the template `ciq` on the disposable docker PG at 127.0.0.1:55432 (Redis at 127.0.0.1:56379), with `BATCH_STOP_RULE_FAILED_JOBS=2`. Production `companyiq_v3` and the hosted `default` DB were not used.
- **Seed.** 1 completed company with scores and 3 jobs that fail; one of them is still queued in BullMQ.
- **Below threshold.** The real `failJob` writes the first failure. `COUNT_BATCH_FAILURES_SQL` counts 1 failed job, 0 exhausted. The rule does not trip, and the batch is still running.
- **Concurrent race.** Two failures, then two `stopRuleTripped` calls run concurrently:
- Both report stopped, so each suppresses its retry. 0 retries were enqueued.
- `batch_runs.status='cancelled'` and the acceptance state is rejected.
- There was exactly one PG transition; the other `cancelBatchRun` was the idempotent no-op, and the losing path logged “snapshot left to that path”.
- Exactly **one** snapshot row exists. It holds only the completed company and is `rejected` with “cancelled/partial … not comprehensive”. There is no duplicate accepted snapshot.
- The queued BullMQ job was purged, and the Redis cross-replica flag was set.
- `claimJob` refuses every failed job, and their attempts are unchanged, so there is no uncontrolled retry.
- **Late failure after cancel.** A third failure is still stopped, there is still one snapshot, and still zero retries.
- **Teardown.** Deletes the Redis flag, terminates backends, drops the DB, and checks that 0 matching DBs remain. The docker containers `ciq-int-pg` / `ciq-int-redis` are disposable local instances and were left running.

5. tsc

`npx tsc --noEmit -p .` on `9a1b931` gives 62 errors, the same as the base worktree. The sorted error lists match once line/column numbers are removed (`diff` empty; output in `evidence/release-items/tsc.txt`).

- The only processor.ts entry is the existing `pdf-parse` TS7016, which is present on the base too.
- Before commit, a first check showed 63. The extra error was `lifecycle-fence.db.test.ts` typing a variable as `Promise<void>` for `cancelBatchRun` , which now returns `Promise<boolean>` . The annotation was fixed, and `lifecycle-fence.db` was re-run: 165/165.

6. Not verified

- **Production SIGABRT.** Not reproduced locally. The link between the browser-lifetime defect and the production failures is inferred from:
- the local launch-failure mode under TasksMax=400;
- the batch 1276 URL reconciliation: Southern lost 37% of its URLs, all with browser-launch failures (see `evidence/batch1276-url-reconciliation.md`).
- **Not tested on Railway** or at production settings. Production already runs with `WORKER_ISOLATION_ENABLED=true` , so **deploying activates the browser fix immediately**; there is no separate switch for it. Only the stop rule is inert, until `BATCH_STOP_RULE_FAILED_JOBS` is set to N > 0. Rollback means redeploying the previous image/commit (`b676c2d` / the prior production deploy), not disabling isolation (see REVIEW_PACKAGE “Rollback procedure”).
- **Whether the fix eliminates production child-start failures** is not shown. It removes the browser accumulation and the related extraction failures in the constrained local repro.
- **No real web fetches or LLM providers.** Repro and integration use locally served PDFs and stubbed providers.
- **Unconfirmed-close path.** The permit is withheld and the browser circuit opened. This is covered by unit logic only; it did not occur in any run (quarantined 0).
- **Stop-rule SQL and the cancel path** were run against a disposable local PG only (§4), not against a production-like dataset or at production concurrency.